AET 5420 Project 1

Linear System Modeling using Impulse Response

Austin Arias, Kritan Duwal, Sihyeon Park

1. Introduction

In audio signal processing, the concept of an Impulse Response (IR) provides a fundamental framework for understanding how systems shape sound. An IR represents how a system reacts to an ideal impulse containing all frequencies, effectively capturing both its time-domain behavior, such as delay and decay, and its frequency-dependent characteristics. Through convolution, any input signal can be processed using this response to predict how the system would alter it. This principle enables the modeling of digital effects, acoustic spaces, and hardware processors using measured impulse responses.

This project investigates how accurately a system's actual behavior can be replicated using its IR and convolution. By comparing outputs from real systems with those generated through convolution, differences in amplitude, phase, and practical limitations can be observed. Digital systems are expected to closely match their modeled outputs, while physical systems may introduce deviations due to environmental and hardware factors. MATLAB is used as the primary analysis tool to compute IRs, perform convolution, and evaluate discrepancies, ultimately providing a structured comparison between measured systems and their reconstructed counterparts.

2. Methods

2.1. Part 1 - Direct IR Method

2.1.1. Impulse Generation and Digital Processing

An impulse signal was generated in MATLAB at a sampling rate of 48 kHz to serve as the excitation source for the system measurement. The signal consisted of a one-second vector with a single unit sample at the first index and zeros elsewhere, approximating a discrete-time impulse. This signal was used in Pro Tools to measure the IRs of two digital systems: a 7-band equalizer (Figure 1) and a delay effect (Figure 2). The processed outputs were recorded (Figure 3) and exported as audio files representing the impulse responses of each system. These measured impulse responses were then imported into MATLAB for further analysis. This process allowed the characteristics of each digital system to be captured in both the time and frequency domains.

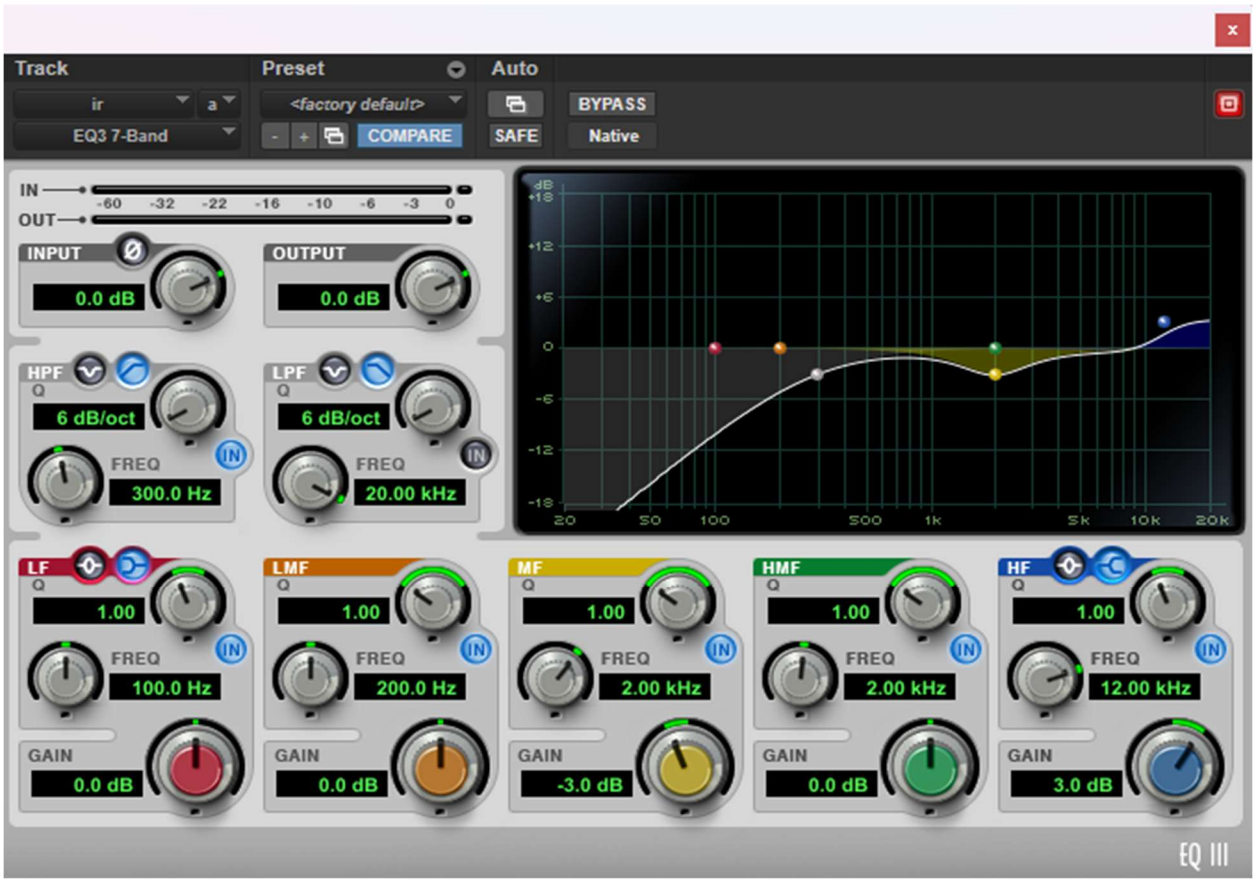


Figure 1: 7-band EQ settings: HPF at 300Hz, 3.0dB Mid Cut at 2kHz, and 3.0dB High Shelf at 12kHz.



Figure 2: Mod Delay III settings: Delay time of 250.0ms, Feedback of 50%.

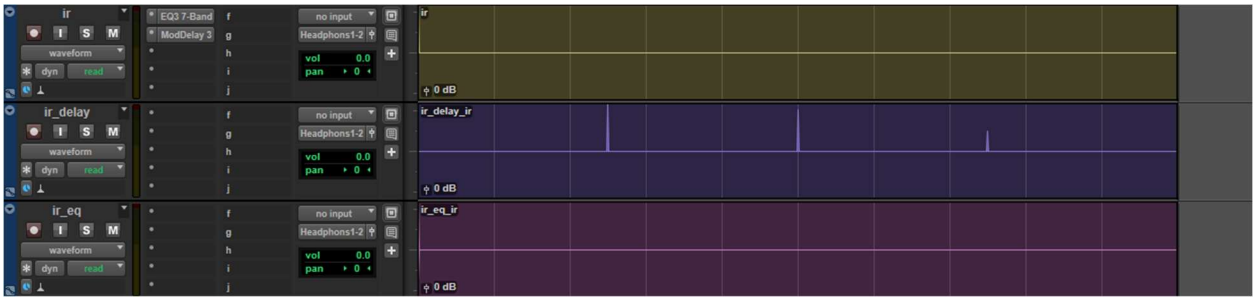


Figure 3: Synthesized impulse signal and the processed IRs with respective digital processing.

2.1.2. Convolution and Null Testing

A dry vocal recording was imported into MATLAB and convolved with each measured IR to simulate the outputs of the digital systems. The convolution operation produced reconstructed signals corresponding to the equalizer and delay processing. In parallel, the same vocal signal was processed directly in Pro Tools using identical plugin settings, and the resulting outputs were recorded as reference signals (Figure 4). To ensure accurate comparison, both the convolution results and the Pro Tools outputs were zero-padded to equal lengths. A null test was then performed by subtracting the convolution outputs from the corresponding processed signals. This procedure allowed the residual signal to represent the difference between the modeled system response and the actual system output, providing a quantitative evaluation of the accuracy of the impulse response-based reconstruction.



Figure 4: Pro Tools processing of vocal.wav file.

2.2. Part 2 - The Sine Sweep Method

2.2.1. Hardware Equalizer Analysis

To measure the response of a hardware EQ, a sine sweep was synthesized in MATLAB. The original sweep file, sweep.wav, was patched through a George Massenburg Labs 8200 (GML 8200) (Figure 6) and recorded into Protools as sweepHardwareEQ.wav. The provided vocal sample, Vocal.wav, was also recorded through the GML 8200 and named vocalHardwareEQ.wav (Figure 7). The

settings for the EQ were as follows: +6 dB at 40 Hz, -6 dB at 8kHz, and +6 dB at 16 kHz. These settings were intended to include both frequency boosting and cutting to the respective test signals.

In MATLAB, the original and recorded sweep were transformed into the frequency domain using FFT. The transfer function of the hardware EQ was estimated using both direct spectral division and Tikhonov regularization. Several regularization values were tested, and lambda (λ) = 1 produced the most stable overall result. The recovered IR was shifted, so the main peak occurred at the beginning, trimmed to a short window (0.15 ms), and then used to generate a magnitude-response plot on a decibel scale with logarithmic frequency axis. The regularized IR was then also convolved with the vocal test signal. For the final null test, the hardware EQ vocal file and the convolved vocal were aligned using cross-correlation before subtraction.

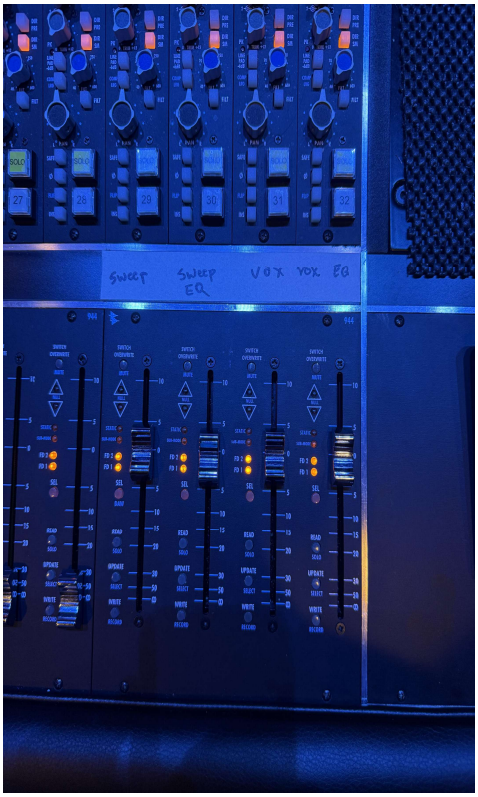


Figure 5: Console desk layout.



Figure 6: EQ settings on GML8200.

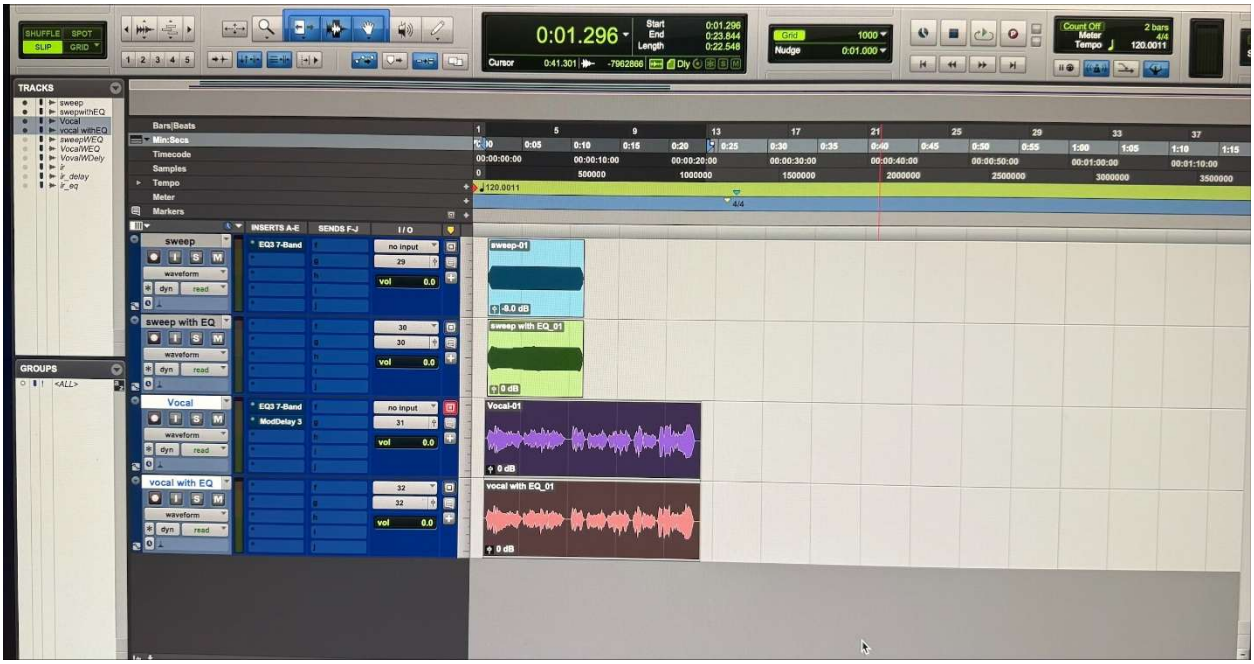


Figure 7: Pro Tools session for Hardware EQ processing.

2.2.2. Acoustic Environment Analysis

The acoustic measurements used for this project had been conducted at Church Street United Methodist Church, located at 900 Henley Street, Knoxville, on June 27, 2025. Impulse response of the space were captured with Room EQ Wizard (REW) which was used to synthesize and play back a sine sweep signal while simultaneously recording the system's response. The measurement setup consisted of three loudspeakers (three full-range speakers and one subwoofer), a Focusrite Scarlett 18i8 audio interface, Earthworks omnidirectional microphones, and a 360-degree camera.

A JavaScript web application (Figure 9) had been developed to display the 360-degree images captured at the venue and to play back any sound file as if the listener were physically present in the space. Using the IR captured at receiver position 1 (Figure 8) from that session ("UMC_R1_IR") together with the provided test signal "Vocal.wav", the web application rendered a new convolved audio file called "UMC_Vocal_R1", simulating how the vocal would sound inside the church.



Figure 8: 360-degree image of position R1 at Church Street United Methodist Church.

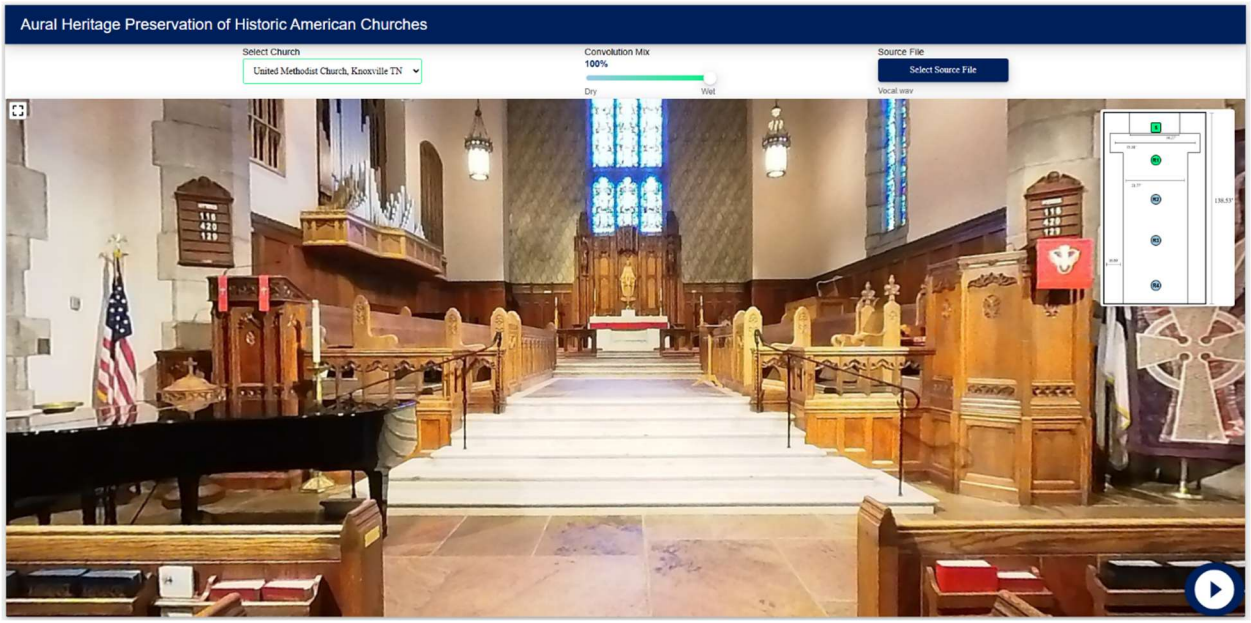


Figure 9: Javascript Web Application Interface.

In MATLAB, all three sound files were imported for analysis. Because the IR had already been captured directly by REW, the deconvolution step was skipped. The IR was plotted in the time domain on a decibel scale using the expression *plot(t, 20*log10(abs(h)))*, allowing the decay behavior of the room to be visualized clearly. The MATLAB built-in function *conv* was then used to convolve the original vocal file with the IR, producing a new processed signal. To prepare for the null test, the recorded audio file was time-aligned with the convolved output using the *xcorr*

(cross-correlation) function, after which the two signals were subtracted from one another to evaluate the difference.

3. Results

3.1. Part 1 - Direct IR Method

3.1.1. Equalizer Impulse Response

The magnitude response of the equalizer IR is shown in Figure 10. The frequency-domain representation reveals a clear attenuation in the low-frequency region, indicating the presence of a high-pass filtering effect. As the frequency increases, the response gradually rises, followed by a noticeable dip in the mid-frequency range around 2 kHz, which corresponds to a midrange cut applied in the equalizer. Beyond this region, the magnitude increases again toward higher frequencies, demonstrating a high-frequency shelf.

These characteristics are consistent with the equalizer settings applied during the measurement process, confirming that the impulse response accurately captures the spectral behavior of the digital EQ system. The smooth variation of the response across frequency further indicates that the system behaves as a linear time-invariant filter, which can be effectively modeled using convolution.

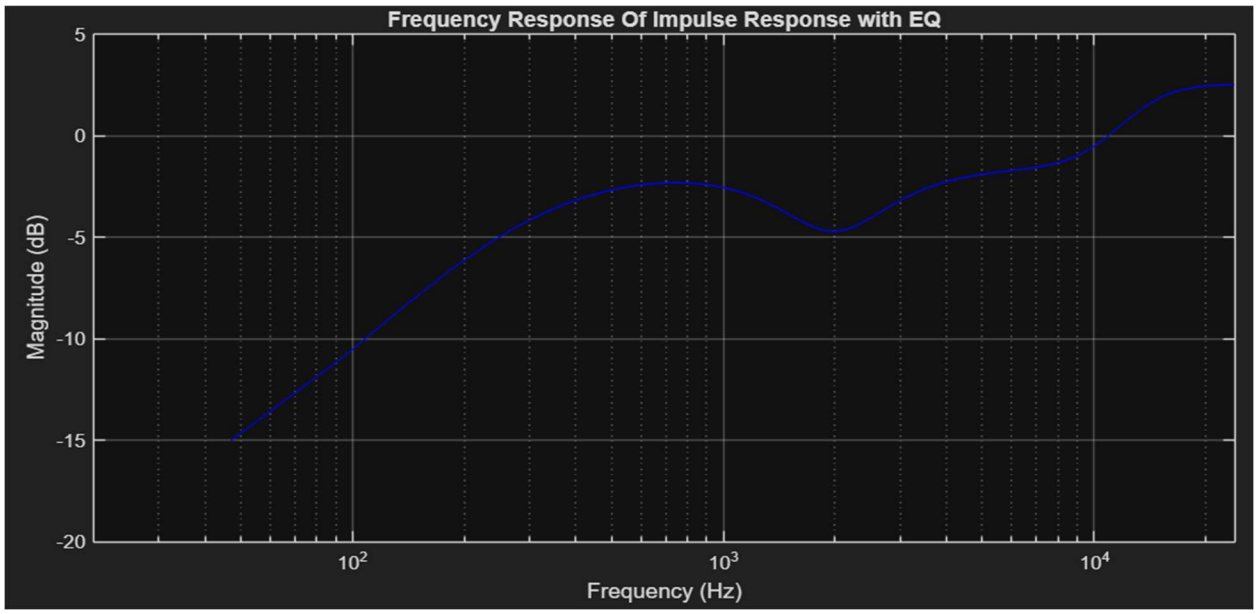


Figure 10: Magnitude Frequency Response of synthesized impulse response with 7-band EQ.

3.1.2. Delay Impulse Response

As shown in Figure 11, the IR of the delay system is represented in the time domain. The plot exhibits a series of discrete impulses following the initial excitation, which correspond to repeated echoes introduced by the delay effect. The temporal spacing between these impulses reflects the delay time parameter, while the decreasing amplitude of successive peaks indicates the effect of feedback attenuation within the system.

The presence of multiple delayed impulses confirms that the system produces recursive echoes, characteristic of a feedback delay structure. The amplitude reduction across repetitions demonstrates energy decay over time, which is consistent with the negative feedback setting applied in the delay plugin. These observations verify that the measured impulse response effectively captures the temporal behavior of the delay system and its echo generation mechanism.

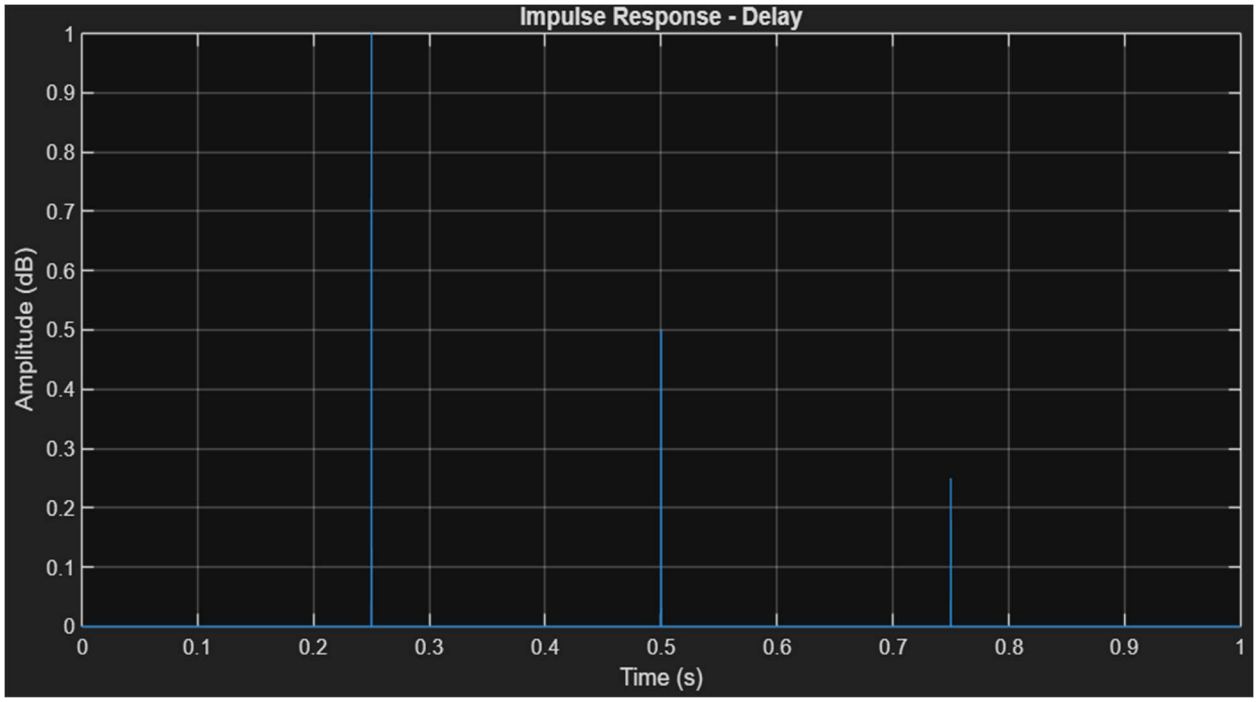


Figure 11: Time-domain plot of IR with delay processing.

3.1.3. Null Tests

The results of the null test for both the equalizer and delay systems are shown in Figures 12 and 13 respectively. In an ideal case, if the impulse response perfectly represents the system, the subtraction between the processed signal and the convolution output would result in complete cancellation, producing a zero-valued signal. However, the residual signals observed in both cases indicate that perfect cancellation was not achieved.

Although the overall waveform shapes of the Pro Tools outputs and the MATLAB convolution results are highly similar, the remaining residual signals suggest small discrepancies between the modeled and actual system responses. These differences may arise from factors such as numerical precision limitations, gain mismatches, or internal processing characteristics of the digital plugins.

Despite the presence of residual signals, the relatively low amplitude (<0.06) of the null test outputs compared to the original signals indicates that the impulse response-based modeling closely approximates the behavior of the digital systems.

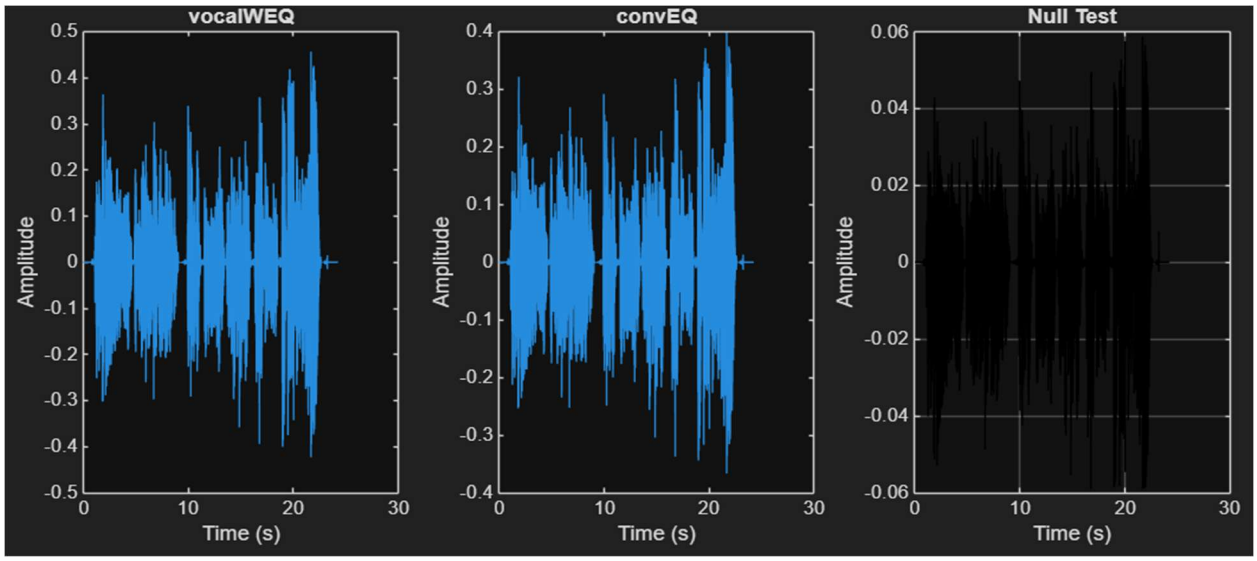


Figure 12: Null Test for 7-band EQ.

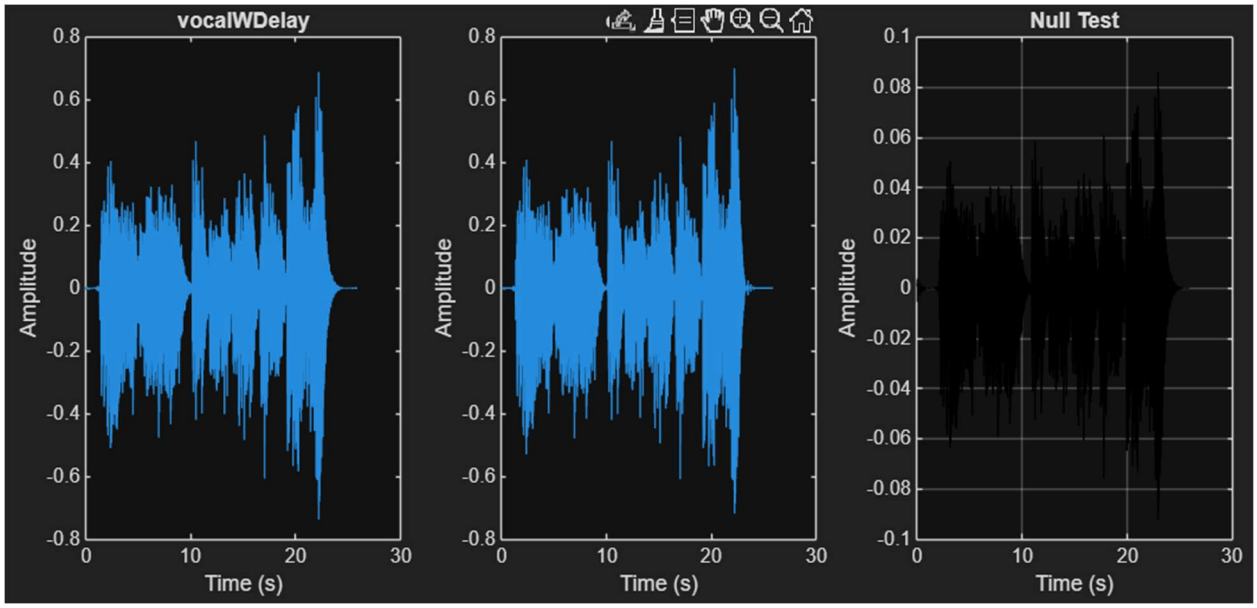


Figure 13: Null Test for digital delay.

3.2. Part 2 - The Sine Sweep Method

3.2.1. Hardware Equalizer Analysis

Direct deconvolution produced instable results (Figure 14). The regularized result with λ = 1 was more stable and used as the recovered response. The magnitude plot showed boosting in the low frequency around 40Hz, and in the high frequencies near 16kHz. It showed a general cut in the 8 kHz frequency range but with a Q value that represented a wide band, the plot did not exactly reproduce what was intended with the dialed settings.

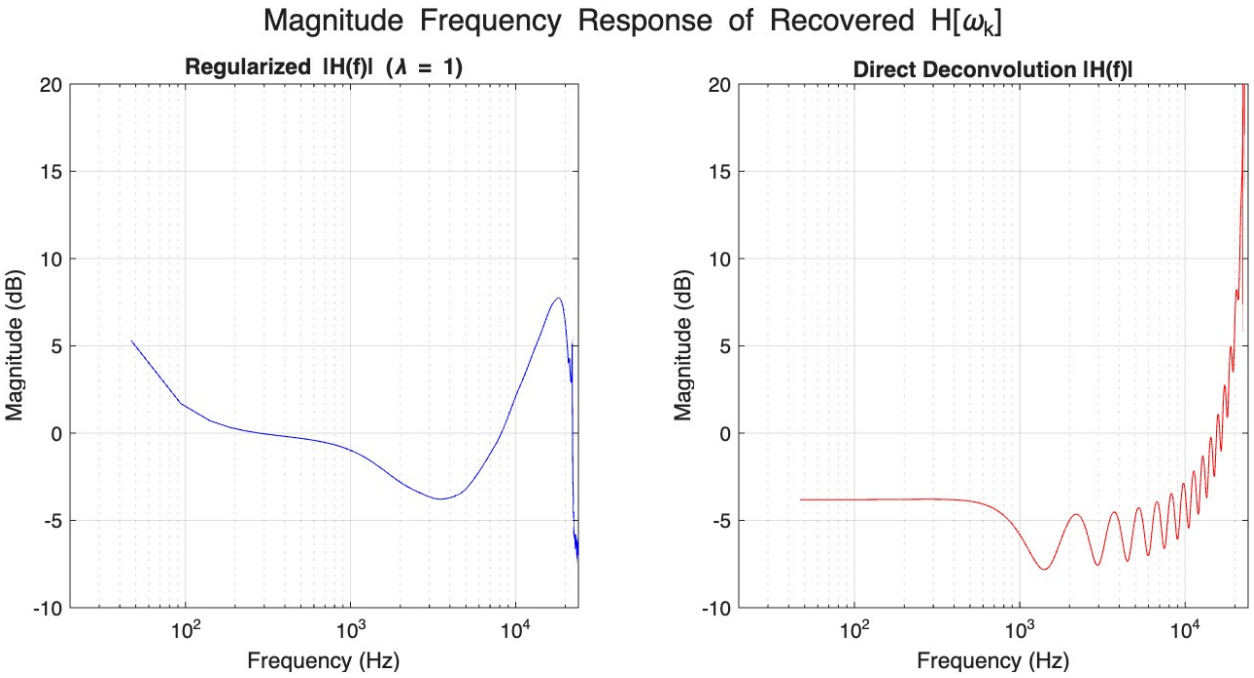


Figure 14: Magnitude Frequency Response of Recovered Impulse Response in MATLAB.

The convolved vocal was then compared to the recorded signal using a null test (Figure 15). After alignment via cross-correlation, the null test improved but did not cancel completely. This is most likely due to the analog nature of the hardware EQ, noise present in the signal, and other imperfections with the physical system.

The sine sweep methods successfully produced a reliable estimate of the hardware response, but the result was less perfect than digital measurements. Regularization was necessary, and λ of 1 resulted in the best measurement result. Although the vocal null test was not zero, this can be expected when using hardware processing and the regularized response still provided a reasonable approximation of the hardware settings.

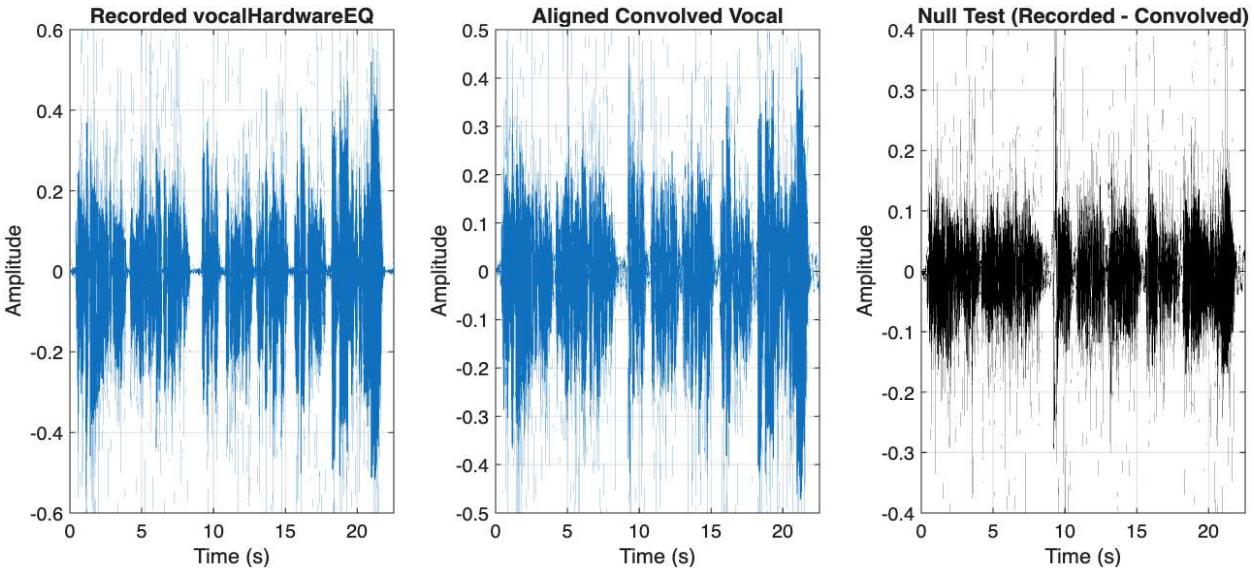


Figure 15: Null Test of Hardware EQ using regularized deconvolved impulse response.

3.2.2. Acoustic Environment Analysis

The time-domain plot of the captured IR is shown in figure 16. We can see that the amplitude of the impulse response decreases by approximately 60dB at around the 1-second mark, corresponding to a reverb time (RT60) of roughly 1 second. While this value may seem low for a church this size, it is consistent with the measurement position used. The IR was captured at receiver position R1, which was located only 10 feet away from the loudspeakers. At such a short distance, the direct sound dominates and the reverberant decay is relatively brief, resulting in a lower RT60 than if it were measured at greater distances from the source.

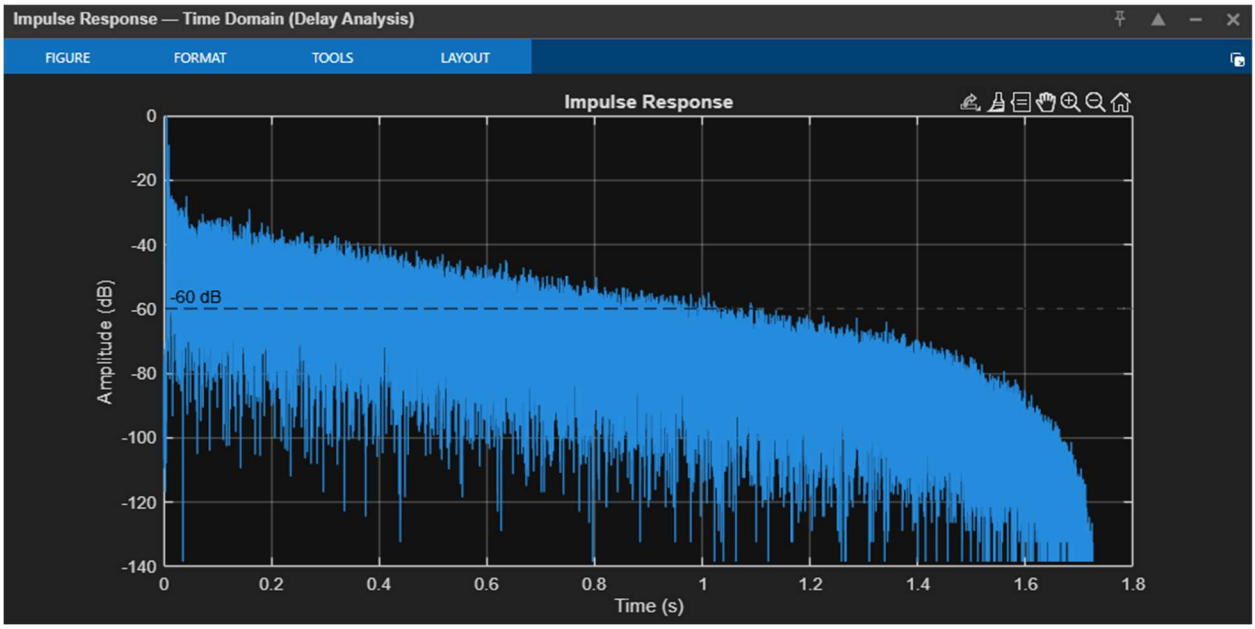


Figure 16: Captured Impulse Response of R1 at Church Street United Methodist Church.

The null test results (Figure 17) revealed a discrepancy between the MATLAB-convolved signal and the recording produced by the JavaScript web application. Without any normalization applied to the convolved MATLAB output, the amplitude of the convolved signal (conv) was approximately four times larger than that of the recorded file (y). This amplitude difference can be attributed to the fact that the JavaScript web application normalizes its convolved output prior to playback, whereas the raw MATLAB convolution output retains its full unnormalized amplitude.

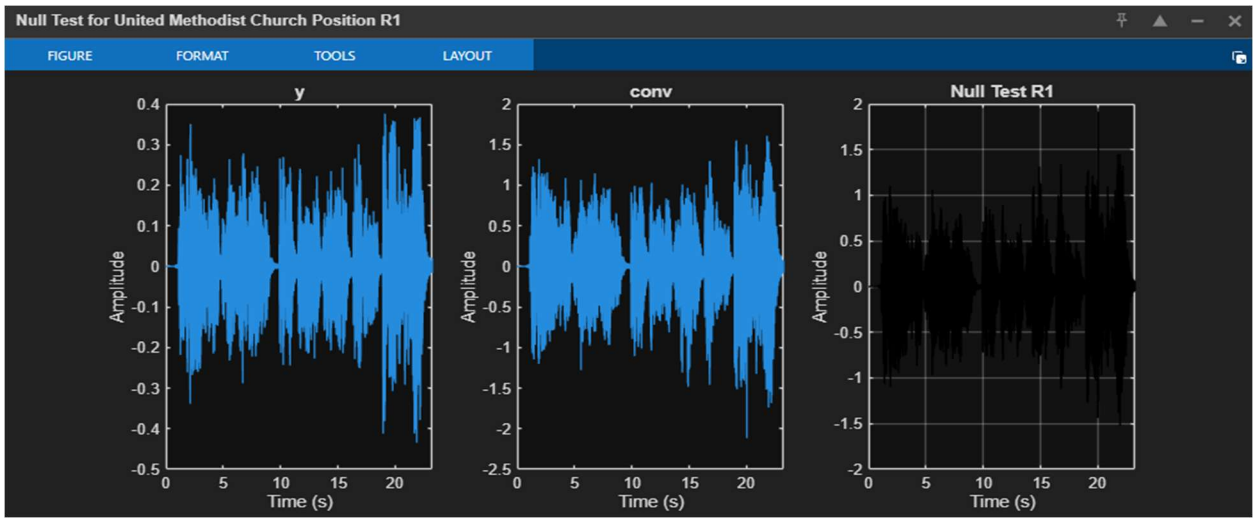


Figure 17: Null Test before normalizing the convolved output of MATLAB.

However, even after normalizing the convolved MATLAB output, the convolved signal remained approximately twice the amplitude of the recorded file (Figure 18). The precise cause of this remaining discrepancy is not fully understood, though it may have been from additional gain processing or limiting applied during playback by the web browser.

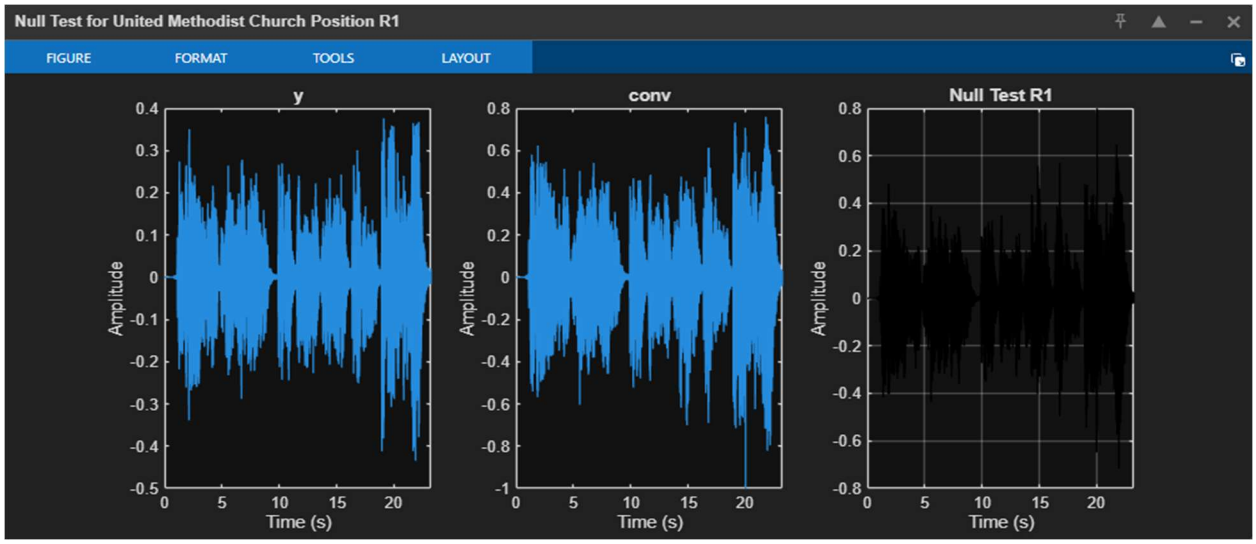


Figure 18: Null Test after normalizing the convolved output of MATLAB.

To compensate for the remaining amplitude mismatch, the normalized convolved output was scaled by a factor of 0.5 before performing the subtraction. The resulting null test plot is shown in Figure 19. Despite the scaling correction, prominent residual signal content remains visible in the null test output rather than the near-zero result expected from two identical signals. This outcome is explained by a fundamental difference between the two processing pipelines: the JavaScript web application blends both a dry signal and a wet signal together to simulate realistic room playback, whereas the MATLAB convolution produces only the fully wet signal with no dry component. As a result, the null test residual captures the dry signal that is present in the web application's output but absent from the MATLAB result, along with any additional differences introduced by the amplitude scaling during normalization.

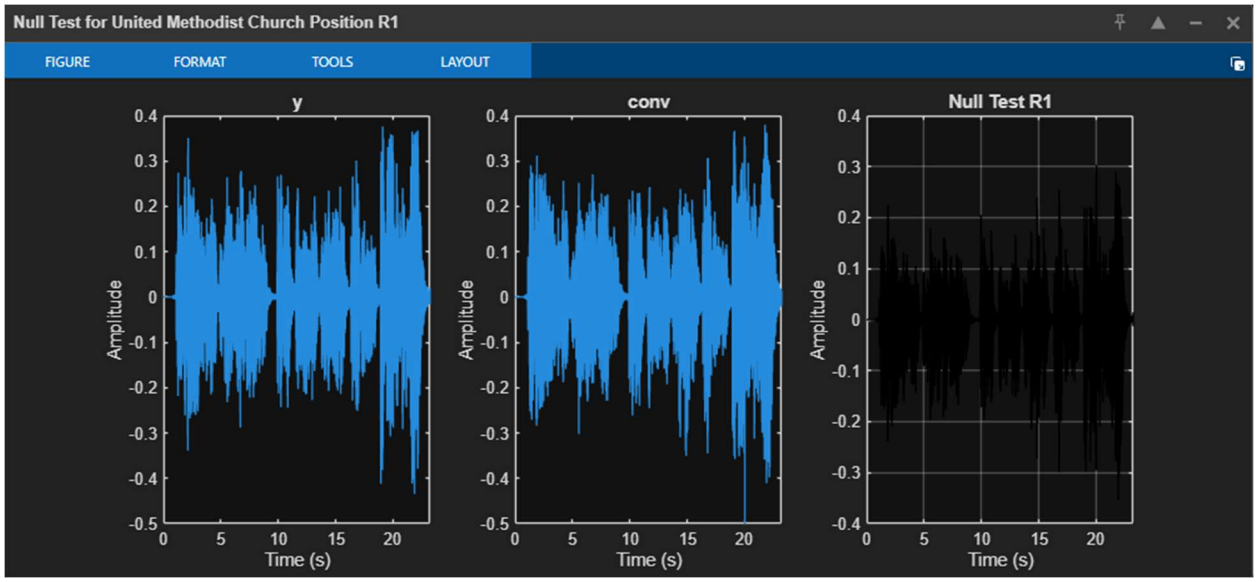


Figure 19: Null Test after normalizing and scaling the convolved output of MATLAB.

4. Conclusion

In summary, this project demonstrated that impulse response measurement and convolution are effective techniques for modeling the behavior of both digital and physical audio systems. Across all the systems examined the convolution-based reconstructions produced outputs that closely resembled their reference counterparts, confirming the theoretical validity of the IR-based approach.

Even though the null test results revealed that perfect signal cancellation was not achieved, for the digital systems, small residual signals remained which may have been due to numerical precision limitations and internal plugin operations such as interpolation or gain scaling that fall outside what a measured IR can fully capture. For the hardware equalizer, the analog nature of

the GML 8200 introduced additional sources of deviation, which might have included thermal noise and nonlinear behavior, making regularization through the Tikhonov method a necessary step for obtaining a stable transfer function estimate.

The acoustic environment analysis further illustrated the complexity of real-world IR modeling. Amplitude mismatches between the MATLAB convolution output and the JavaScript web application's rendered audio highlighted how post-processing decisions such as normalization, wet/dry blending, and browser-level gain management can introduce meaningful differences that are entirely separate from the accuracy of the IR itself.

5. Appendix

```
% Direct IR Analysis
clc; clearvars; close all;

% synthesize impulse response
Fs = 48000;
Ts = 1/Fs;
durSec = 1;
N = durSec * Fs;

% Create impulse signal
ir = zeros(N, 1);
ir(1) = 1;

% Load measured IR files
[ir_eq, Fs_eq] = audioread('ir_eq.wav');
[ir_delay, Fs_delay] = audioread('ir_delay.wav');

if size(ir_delay, 2) > 1
    ir_delay = mean(ir_delay, 2);
end

% Plot EQ magnitude response
[H_fz, f] = freqz(ir_eq, 1, 4096, Fs);

figure('Name', 'Magnitude Frequency Response', ...
       'NumberTitle', 'off', ...
       'Position', [100 100 1000 450]);
semilogx(f, 20*log10(abs(H_fz) + eps), 'b', 'LineWidth', 1.2);
title('Frequency Response of EQ Impulse Response');
xlabel('Frequency (Hz)');
ylabel('Magnitude (dB)');
xlim([20 Fs/2]);
grid on;

% Plot delay IR in time domain
t_delay = (0:length(ir_delay)-1) * Ts;

figure('Name', 'Impulse Response - Delay', ...
       'NumberTitle', 'off', ...
       'Position', [100 100 1000 450]);
plot(t_delay, ir_delay, 'LineWidth', 1.0);
title('Delay Impulse Response');
xlabel('Time (s)');
```

```
ylabel('Amplitude');
grid on;

% Load vocal file
[vocal, Fs_vocal] = audioread('Vocal.wav');

if size(vocal, 2) > 1
    vocal = mean(vocal, 2);
end

if Fs_vocal ~= Fs
    error('Vocal.wav sample rate does not match 48 kHz.');
end

% Convolve vocal with IRs
convEQ = conv(vocal, ir_eq);
convDelay = conv(vocal, ir_delay);

% Save convolution results
audiowrite('vocal_eq_conv.wav', convEQ, Fs);
audiowrite('vocal_delay_conv.wav', convDelay, Fs);

% Load Pro Tools processed vocal files
[vocalWEQ, Fs_vocalWEQ] = audioread('VocalWEQ.wav');
[vocalWDelay, Fs_vocalWDelay] = audioread('VocalWDelay.wav');

if size(vocalWEQ, 2) > 1
    vocalWEQ = mean(vocalWEQ, 2);
end

if size(vocalWDelay, 2) > 1
    vocalWDelay = mean(vocalWDelay, 2);
end

if Fs_vocalWEQ ~= Fs || Fs_vocalWDelay ~= Fs
    error('Processed vocal file sample rates do not match 48 kHz.');
end

% ----- Null test for EQ -----
[c_eq, lags_eq] = xcorr(vocalWEQ, convEQ);
[~, idx_eq] = max(abs(c_eq));
lag_eq = lags_eq(idx_eq);

if lag_eq > 0
    vocalWEQ_aligned = vocalWEQ(1 + lag_eq:end);
```

```
    convEQ_aligned = convEQ;
else
    vocalWEQ_aligned = vocalWEQ;
    convEQ_aligned = convEQ(1 - lag_eq:end);
end

N_eq = min(length(vocalWEQ_aligned), length(convEQ_aligned));
vocalWEQ_aligned = vocalWEQ_aligned(1:N_eq);
convEQ_aligned = convEQ_aligned(1:N_eq);

nullTestEQ = vocalWEQ_aligned - convEQ_aligned;
t_nullEQ = (0:N_eq-1) / Fs;

figure('Name', 'Null Test for EQ', ...
       'NumberTitle', 'off', ...
       'Position', [100 100 1100 400]);

subplot(1,3,1);
plot(t_nullEQ, vocalWEQ_aligned);
title('Pro Tools EQ Output');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;

subplot(1,3,2);
plot(t_nullEQ, convEQ_aligned);
title('MATLAB Convolution Output');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;

subplot(1,3,3);
plot(t_nullEQ, nullTestEQ, 'k');
title('EQ Null Test');
xlabel('Time (s)');
ylabel('Residual');
grid on;

% ----- Null test for Delay -----
[c_delay, lags_delay] = xcorr(vocalWDelay, convDelay);
[~, idx_delay] = max(abs(c_delay));
lag_delay = lags_delay(idx_delay);

if lag_delay > 0
    vocalWDelay_aligned = vocalWDelay(1 + lag_delay:end);
```

```
    convDelay_aligned = convDelay;
else
    vocalWDelay_aligned = vocalWDelay;
    convDelay_aligned = convDelay(1 - lag_delay:end);
end

N_delay = min(length(vocalWDelay_aligned), length(convDelay_aligned));
vocalWDelay_aligned = vocalWDelay_aligned(1:N_delay);
convDelay_aligned = convDelay_aligned(1:N_delay);

nullTestDelay = vocalWDelay_aligned - convDelay_aligned;
t_nullDelay = (0:N_delay-1) / Fs;

% Make common x-axis length
N_common = min(N_eq, N_delay);

% Trim EQ
vocalWEQ_aligned = vocalWEQ_aligned(1:N_common);
convEQ_aligned   = convEQ_aligned(1:N_common);
nullTestEQ       = nullTestEQ(1:N_common);

% Trim Delay
vocalWDelay_aligned = vocalWDelay_aligned(1:N_common);
convDelay_aligned   = convDelay_aligned(1:N_common);
nullTestDelay       = nullTestDelay(1:N_common);

% New time axis
t_common = (0:N_common-1) / Fs;

% plot
figure('Name', 'Null Test for EQ', ...
       'NumberTitle', 'off', ...
       'Position', [100 100 1100 400]);

subplot(1,3,1);
plot(t_common, vocalWEQ_aligned);
title('Pro Tools EQ Output');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;

subplot(1,3,2);
plot(t_common, convEQ_aligned);
title('MATLAB Convolution Output');
xlabel('Time (s)');
```

```
ylabel('Amplitude');
grid on;

subplot(1,3,3);
plot(t_common, nullTestEQ, 'k');
title('EQ Null Test');
xlabel('Time (s)');
ylabel('Residual');
grid on;

figure('Name', 'Null Test for Delay', ...
       'NumberTitle', 'off', ...
       'Position', [100 100 1100 400]);

subplot(1,3,1);
plot(t_common, vocalWDelay_aligned);
title('Pro Tools Delay Output');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;

subplot(1,3,2);
plot(t_common, convDelay_aligned);
title('MATLAB Convolution Output');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;

subplot(1,3,3);
plot(t_common, nullTestDelay, 'k');
title('Delay Null Test');
xlabel('Time (s)');
ylabel('Residual');
grid on;
```

```
% Synthesize a sine sweep signal in Matlab

clc; clear; close all;

Fs   = 48000;
Ts   = 1/Fs;

% 10 second sweep
t = (0 : Ts : 10)';

%x = chirp(t, 0, 10, 22000);
%sound(x,Fs);

% Fade duration (seconds)
fade_dur = 0.5;

% number of fade samples
N_fade = round(fade_dur * Fs);
N_total  = length(t);

% Linear ramp: rises 0 → 1 over N_fade points
lin_ramp = linspace(0, 1, N_fade)';

% Build full envelope: fade-in | ones | fade-out
envelope = ones(N_total, 1);
envelope(1 : N_fade) = lin_ramp;            % fade-in
envelope(N_total-N_fade+1 : N_total) = flipud(lin_ramp);    % fade-out

% built-in sweep synthesizer
% Linear sweep, 0° initial phase (default)
x_lin_0 = chirp(t, 0, 10, 22000);   % phi0 = 0°

% Apply fade envelope
x_lin_0_faded   = x_lin_0(:)   .* envelope;

% Write WAV file
%audiowrite('sweep.wav', x_lin_0_faded, Fs);

%sound(x_lin_0_faded,Fs);
plot(t,x_lin_0_faded);
```

```
% hardwareEQAnalysis_final.m
%
% Austin, Kritan, Sihyeon
%
% AET 5420 Project 1
% Hardware Equalizer IR Analysis
clear; clc; close all;
%% Import sweep files
[x, Fs_x] = audioread("sweep.wav");
[y, Fs_y] = audioread("sweepHardwareEQ.wav");
% Convert to mono if needed
if size(x,2) > 1
x = mean(x,2);
end
if size(y,2) > 1
y = mean(y,2);
end
% Check sample rates
if Fs_x ~= Fs_y
error('Sample rates do not match between sweep.wav and sweepHardwareEQ.wav.');
end
Fs = Fs_x;
Ts = 1 / Fs;
%% Match lengths by zero-padding
Nx = length(x);
Ny = length(y);
N = max(Nx, Ny);
x = [x; zeros(N - Nx, 1)];
y = [y; zeros(N - Ny, 1)];
%% Convert to frequency domain
X = fft(x, N);
Y = fft(y, N);
%% Regularized deconvolution
lambda = 1;
H_reg = zeros(N, 1);
for k = 1:N
Xk = X(k);
Yk = Y(k);
Xk_conj = conj(Xk);
H_reg(k) = (Xk_conj * Yk) / ((Xk_conj * Xk + lambda) * Ts);
end
%% Direct deconvolution for comparison
X_safe = X;
X_safe(abs(X_safe) < eps) = eps;
H_direct = Y ./ X_safe;
```

```
%% Convert H[wk] to h[n]
h_reg = real(ifft(H_reg));
h_direct = real(ifft(H_direct));
%% Shift peak to beginning
[~, peak_idx_h_reg] = max(abs(h_reg));
h_reg = circshift(h_reg, -(peak_idx_h_reg - 1));
[~, peak_idx_h_direct] = max(abs(h_direct));
h_direct = circshift(h_direct, -(peak_idx_h_direct - 1));
%% Trim recovered IR
ir_length_sec = 0.15;
ir_length_samples = round(ir_length_sec * Fs);
h_reg = h_reg(1:min(ir_length_samples, N));
h_direct = h_direct(1:min(ir_length_samples, N));
%% Normalize IRs for stable plotting/convolution
h_reg = h_reg / max(abs(h_reg));
h_direct = h_direct / max(abs(h_direct));
%% Magnitude response plot
[H_fz_reg, W_reg] = freqz(h_reg);
[H_fz_direct, W_direct] = freqz(h_direct);
f_reg = W_reg / (2*pi) * Fs;
f_direct = W_direct / (2*pi) * Fs;
figure('Name','Magnitude Frequency Response (freqz)', ...
'NumberTitle','off', 'Position',[100 100 1000 450]);
subplot(1,2,1)
semilogx(f_reg, 20*log10(abs(H_fz_reg) + eps), 'b');
title(sprintf('Regularized |H(f)| (\\lambda = %g)', lambda));
xlabel('Frequency (Hz)');
ylabel('Magnitude (dB)');
xlim([20 Fs/2]);
ylim([-10 20]);
grid on;
subplot(1,2,2)
semilogx(f_direct, 20*log10(abs(H_fz_direct) + eps), 'r');
title('Direct Deconvolution |H(f)|');
xlabel('Frequency (Hz)');
ylabel('Magnitude (dB)');
xlim([20 Fs/2]);
ylim([-10 20]);
grid on;
sgtitle('Magnitude Frequency Response of Recovered H[\omega_k]');
%% Import vocal files
[vocal, Fs_vocal] = audioread("Vocal.wav");
[vocalWEQ, Fs_vocalWEQ] = audioread("vocalHardwareEQ.wav");
% Convert to mono if needed
if size(vocal,2) > 1
```

```
vocal = mean(vocal,2);
end
if size(vocalWEQ,2) > 1
vocalWEQ = mean(vocalWEQ,2);
end
% Check sample rates
if Fs_vocal ~= Fs || Fs_vocalWEQ ~= Fs
error('Sample rates do not match vocal files.');
end
%% Convolve using recovered IR
convVocalEQ = conv(vocal, h_reg);
%% Null test using h_reg and xcorr alignment
[c_delay, lags_delay] = xcorr(vocalWEQ, convVocalEQ);
[~, idx_delay] = max(abs(c_delay));
lag_delay = lags_delay(idx_delay);
if lag_delay > 0
y_aligned = vocalWEQ(1 + lag_delay:end);
conv_aligned = convVocalEQ;
else
y_aligned = vocalWEQ;
conv_aligned = convVocalEQ(1 - lag_delay:end);
end
N_delay = min(length(y_aligned), length(conv_aligned));
y_aligned = y_aligned(1:N_delay);
conv_aligned = conv_aligned(1:N_delay);
% Optional level match
alpha = (y_aligned' * conv_aligned) / (conv_aligned' * conv_aligned + eps);
conv_aligned = alpha * conv_aligned;
nullTest = y_aligned - conv_aligned;
t_null = (0:N_delay-1) / Fs;
figure('Name','Hardware EQ Null Test using h-reg', ...
'NumberTitle','off','Position',[100 100 1000 400]);
subplot(1,3,1);
plot(t_null, y_aligned);
title('Recorded vocalHardwareEQ');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;
subplot(1,3,2);
plot(t_null, conv_aligned);
title('Aligned Convolved Vocal');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;
subplot(1,3,3);
```

```
plot(t_null, nullTest, 'k');
title('Null Test (Recorded - Convolved)');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;
%% Compare using h_direct
convVocalEQ_direct = conv(vocal, h_direct);
[c_delay, lags_delay] = xcorr(vocalWEQ, convVocalEQ_direct);
[~, idx_delay] = max(abs(c_delay));
lag_delay = lags_delay(idx_delay);
if lag_delay > 0
y_aligned = vocalWEQ(1 + lag_delay:end);
conv_aligned = convVocalEQ_direct;
else
y_aligned = vocalWEQ;
conv_aligned = convVocalEQ_direct(1 - lag_delay:end);
end
N_delay = min(length(y_aligned), length(conv_aligned));
y_aligned = y_aligned(1:N_delay);
conv_aligned = conv_aligned(1:N_delay);
% Optional level match
alpha = (y_aligned' * conv_aligned) / (conv_aligned' * conv_aligned + eps);
conv_aligned = alpha * conv_aligned;
nullTest = y_aligned - conv_aligned;
t_null = (0:N_delay-1) / Fs;
figure('Name','Hardware EQ Null Test using h-direct', ...
'NumberTitle','off','Position',[100 100 1000 400]);
subplot(1,3,1);
plot(t_null, y_aligned);
title('Recorded vocalHardwareEQ');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;
subplot(1,3,2);
plot(t_null, conv_aligned);
title('Aligned Convolved Vocal');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;
subplot(1,3,3);
plot(t_null, nullTest, 'k');
title('Null Test (Recorded - Convolved)');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;
```

```
% NullTest.m
%
% Austin, Kritan, Sihyeon
%
% AET 5420 Project 1
% Hardware EQ Null Test Using xcorr Alignment
clear; clc; close all;
%% Import files
[recorded, Fs1] = audioread('vocalHardwareEQ.wav');
[alignedConv, Fs2] = audioread('convVocalHardwareEQ_reg_aligned.wav');
% Convert to mono if needed
if size(recorded,2) > 1
recorded = mean(recorded,2);
end
if size(alignedConv,2) > 1
alignedConv = mean(alignedConv,2);
end
% Check sample rates
if Fs1 ~= Fs2
error('Sample rates do not match.');
end
Fs = Fs1;
%% Optional: remove DC offset
recorded = recorded - mean(recorded);
alignedConv = alignedConv - mean(alignedConv);
%% Null test using xcorr alignment
[c_delay, lags_delay] = xcorr(recorded, alignedConv);
[~, idx_delay] = max(abs(c_delay));
lag_delay = lags_delay(idx_delay);
if lag_delay > 0
y_aligned = recorded(1 + lag_delay:end);
convR1_aligned = alignedConv;
else
y_aligned = recorded;
convR1_aligned = alignedConv(1 - lag_delay:end);
end
N_delay = min(length(y_aligned), length(convR1_aligned));
y_aligned = y_aligned(1:N_delay);
convR1_aligned = convR1_aligned(1:N_delay);
% Optional gain match
alpha = (y_aligned' * convR1_aligned) / (convR1_aligned' * convR1_aligned + eps);
convR1_aligned = alpha * convR1_aligned;
% Subtract one waveform from the other
nullTest = y_aligned - convR1_aligned;
%% Plot
```

```
t_null = (0:N_delay-1)' / Fs;
figure('Name','Hardware EQ Null Test','NumberTitle','off','Position',[100 100
1000 400]);
subplot(1,3,1);
plot(t_null, y_aligned);
title('Recorded vocalHardwareEQ');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;
subplot(1,3,2);
plot(t_null, convR1_aligned);
title('Aligned Convolved Vocal');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;
subplot(1,3,3);
plot(t_null, nullTest, 'k');
title('Null Test (Recorded - Convolved)');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;
```

```
%% Acoustic room IR Analysis

clc; clear; close all;

%% import original sound files
[x, Fs] = audioread("Vocal.wav");
[h] = audioread("UMC_R1_IR.wav");
[y] = audioread("UMC_Vocal_R1.wav");

N = length(x);

% plot of the impulse response amplitude to decrease by 60 dB
t = (0:length(h)-1)/Fs;
figure('Name','Impulse Response – Time Domain (Delay Analysis)',
'NumberTitle','off','Position',[100 100 1000 450]);

plot(t,20*log10(abs(h)));
title('Impulse Response');
xlabel('Time (s)'); ylabel('Amplitude (dB)');
yline(-60, 'k--', '-60 dB', 'LabelHorizontalAlignment','left');
grid on;

% convolve using MATLAB conv function
convR1 = conv(x, h);

% normalized convolved output to roughly match the amplitude of y
convR1 = 0.5 * convR1 / max(abs(convR1));

% playback convolved output
%sound(convR1,Fs);

% export convolved output
%audiowrite("convR1.wav", convR1, Fs);

%% Null test: subtract convolved outputs from y for R1

% use correlation to match the two waveforms
[c_delay, lags_delay] = xcorr(y, convR1);
[~, idx_delay] = max(abs(c_delay));
lag_delay = lags_delay(idx_delay);

if lag_delay > 0
    y_aligned = y(1 + lag_delay:end);
    convR1_aligned = convR1;
else
```

```
    y_aligned = y;
    convR1_aligned = convR1(1 - lag_delay:end);
end

N_delay = min(length(y_aligned), length(convR1_aligned));
y_aligned = y_aligned(1:N_delay);
convR1_aligned = convR1_aligned(1:N_delay);

% subtract one waveform from the other
nullTest = y_aligned - convR1_aligned;

% playback/export nulltest
%sound(nullTest,Fs);
%audiowrite("nullTest.wav", nullTest, Fs);

% plot
t_null = (0:N_delay-1) / Fs;

figure('Name','Null Test for United Methodist Church Position
R1','NumberTitle','off','Position',[100 100 1000 400]);

subplot(1,3,1);
plot(t_null,y_aligned);
title('y');
xlabel('Time (s)'); ylabel('Amplitude');
subplot(1,3,2);
plot(t_null,convR1_aligned);
title('conv');
xlabel('Time (s)'); ylabel('Amplitude');
subplot(1,3,3);
plot(t_null, nullTest, 'k');
title('Null Test R1');
xlabel('Time (s)'); ylabel('Amplitude');
grid on;
```